

# Hochschule für Technik und Wirtschaft Berlin

Fachbereich 4  
Campus Wilhelminenhof  
Wilhelminenhofstraße 75A  
12459 Berlin  
<https://www.f4.htw-berlin.de>



Independent Coursework II

## Modifikation von "Quake" mit RTX Remix

Ruslan Novikov

Matrikelnummer: 566919  
03.04.2024

Betreut von  
Prof. Dr. Tobias Lenz



Hiermit erkläre ich, dass ich diese Arbeit selbst verfasst habe, unter ausschließlicher Verwendung der genannten Literatur und Hilfsmittel.

Berlin, 01.04.2024

.....  
(*Unterschrift*)





# Inhaltsverzeichnis

|                                                                       |            |
|-----------------------------------------------------------------------|------------|
| <b>Abbildungsverzeichnis</b>                                          | <b>vii</b> |
| <b>Tabellenverzeichnis</b>                                            | <b>ix</b>  |
| <b>1 Einleitung</b>                                                   | <b>1</b>   |
| 1.1 Motivation und Ziel . . . . .                                     | 1          |
| 1.2 Rahmen . . . . .                                                  | 1          |
| <b>2 Grundlagen</b>                                                   | <b>3</b>   |
| 2.1 Software . . . . .                                                | 3          |
| 2.1.1 RTX Remix . . . . .                                             | 3          |
| 2.1.2 Quake Engine und Source Ports . . . . .                         | 3          |
| <b>3 Schwerpunkte</b>                                                 | <b>5</b>   |
| 3.1 Anforderungen and Kompatibilität . . . . .                        | 5          |
| 3.1.1 Kompatibilität: Ursprüngliche Modifikation . . . . .            | 5          |
| 3.1.2 Kompatibilität: Remix Modifikation . . . . .                    | 5          |
| 3.2 Texturen . . . . .                                                | 6          |
| 3.2.1 Texturen: Remix Modifikation . . . . .                          | 6          |
| 3.2.2 Texturen: Ursprüngliche Modifikation . . . . .                  | 9          |
| 3.3 Kamera . . . . .                                                  | 11         |
| 3.4 Lichtquellen . . . . .                                            | 12         |
| 3.4.1 Lichtquellen: Ursprüngliche Modifikation . . . . .              | 12         |
| 3.4.2 Lichtquellen: Remix Modifikation . . . . .                      | 15         |
| 3.5 Culling & Beschleunigungsstrukturen . . . . .                     | 16         |
| 3.5.1 Culling: Ursprüngliche Modifikation . . . . .                   | 17         |
| 3.5.2 Culling: Remix Modifikation . . . . .                           | 17         |
| 3.5.3 Beschleunigungsstrukturen: Ursprüngliche Modifikation . . . . . | 18         |
| 3.5.4 Beschleunigungsstrukturen: Remix Modifikation . . . . .         | 20         |
| 3.6 Materialien . . . . .                                             | 21         |
| 3.6.1 Materialien: Ursprüngliche Modifikation . . . . .               | 21         |
| 3.6.2 Materialien: Remix Modifikation . . . . .                       | 21         |
| <b>4 Zusammenfassung</b>                                              | <b>25</b>  |
| <b>Literaturverzeichnis</b>                                           | <b>27</b>  |



# Abbildungsverzeichnis

|      |                                                                                              |    |
|------|----------------------------------------------------------------------------------------------|----|
| 3.1  | User Interface Texturen . . . . .                                                            | 7  |
| 3.2  | Mesh Texture und dazugehöriges Mesh (blauer Rahmen) . . . . .                                | 8  |
| 3.3  | Nicht kategorisiert . . . . .                                                                | 9  |
| 3.4  | Kategorie: 'Lightmap texture' . . . . .                                                      | 9  |
| 3.5  | Kategorie: 'Ignore texture' . . . . .                                                        | 10 |
| 3.6  | Emissive Lichttexturen . . . . .                                                             | 10 |
| 3.7  | Ungültige Kamerawerte . . . . .                                                              | 11 |
| 3.8  | Vergleich Aktivierungsfunktion . . . . .                                                     | 13 |
| 3.9  | Lichteinfluss mit (oben) und ohne (unten) Lighttracing bei kleiner Licht-<br>quelle. . . . . | 14 |
| 3.10 | Lichtquellen und deren Eigenschaften . . . . .                                               | 15 |
| 3.11 | Emissive Texturen als Lichtquelle . . . . .                                                  | 17 |
| 3.12 | Unveränderte Geometrie . . . . .                                                             | 18 |
| 3.13 | Culled PVS Geometrie . . . . .                                                               | 19 |
| 3.14 | Statische BLAS . . . . .                                                                     | 19 |
| 3.15 | Dynamische BLAS . . . . .                                                                    | 20 |
| 3.16 | Berechnung des random uniform hemisphere samplings . . . . .                                 | 21 |
| 3.17 | KI-Generierte Diffuse, Normal und Roughness Maps . . . . .                                   | 22 |
| 3.18 | Standard Material . . . . .                                                                  | 22 |
| 3.19 | Material überschrieben mit KI-Generierten PBR Texturen . . . . .                             | 23 |



# Tabellenverzeichnis

|     |                                |   |
|-----|--------------------------------|---|
| 3.1 | Kompatibilitätsliste . . . . . | 6 |
|-----|--------------------------------|---|



# 1 Einleitung

## 1.1 Motivation und Ziel

Im Zuge meiner Bachelorarbeit im Jahr 2022 unternahm ich den Versuch, einen hardwarebeschleunigten Path-Tracing-Renderer mithilfe einer entsprechenden Vulkan-API-Erweiterung in das Spiel 'Quake' zu integrieren [Nov22]. Dies erforderte die Modifizierung großer Teile des ursprünglichen Renderers und ihre Ersetzung durch einen Path-Tracing-Renderer. Zusätzlich waren Anpassungen an anderen Aspekten wie dem Lichtsystem oder dem Cullingsystem erforderlich, um einen reibungslosen Betrieb des Path-Tracing zu gewährleisten. Mittlerweile ermöglichen Tools wie 'RTX Remix' die Modifikation von Spielen zur Nutzung von Path-Tracing, selbst ohne Zugriff auf den Quellcode.

In diesem Zusammenhang plane ich erneut, das Spiel 'Quake' mithilfe von RTX Remix mit einem Path-Tracer auszustatten und dabei die wesentlichen Unterschiede zur ursprünglichen Herangehensweise zu beleuchten.

## 1.2 Rahmen

Für Demonstrationszwecke werde ich Level und Spielszenen von Quake mithilfe der RTX Remix Software modifizieren, wobei ich mich jedoch auf das Hub-Level beschränken werde. Die Erstellung einer vollständig modifizierten und spielbaren Version ist aus Zeitgründen nicht umsetzbar. Zudem werde ich mich auf eine minimale Darstellung von Implementierungsdetails konzentrieren. Das Hauptziel dieser Arbeit besteht darin, den Prozess der Softwarenutzung zu dokumentieren und dabei den Workflow der wichtigsten Aspekte zu vergleichen.

Darüber hinaus beabsichtige ich, sämtliche Spieländerungen ausschließlich mittels der 'RTX Remix Runtime' und der 'RTX Remix Application' vorzunehmen. Auf diese Weise soll die Leistungsfähigkeit des Tools ohne direkten Zugriff auf den Quellcode überprüft werden, sofern dies praktikabel ist. Dieser Ansatz ermöglicht eine detaillierte Identifizierung aktueller Einschränkungen sowie potenzieller Fehlerquellen.





## 2 Grundlagen

### 2.1 Software

#### 2.1.1 RTX Remix

RTX Remix ist eine Open-Source Modding-Plattform, die es ermöglicht, Modelle und Texturen eines Spiels zu ersetzen, während das Lichtsystem des Originalspiels durch einen Path-Tracing-Renderer ersetzt wird [NVI23]. RTX Remix setzt sich aus zwei Hauptkomponenten zusammen:

Die erste Komponente ist die RTX Remix-Applikation, die es Benutzern ermöglicht, Lichter, Texturen und Assets zu modifizieren und hinzuzufügen. Darüber hinaus bietet das Tool die Möglichkeit, Texturen für physikalisch basiertes Rendering (PBR) mithilfe von KI-unterstützten Werkzeugen zu generieren.

Die zweite Komponente ist die RTX Remix-Runtime, eine Laufzeitkomponente, die Aufrufe der zugrunde liegenden Grafik-API abfängt, um Modifikationen mithilfe der oben genannten Remix-Applikation vorzunehmen. Die Runtime selbst ist in zwei Unterkomponenten unterteilt:

Die Remix-Bridge fungiert als Schnittstelle zum Abfangen der API-Aufrufe und dient als Vermittler zwischen der Grafik-API und der Remix-Applikation. Der RTX-Renderer hingegen tauscht die in der Remix-Applikation angepassten Lichter, Texturen und Assets aus und ist zusätzlich für das Path-Tracing verantwortlich.

#### 2.1.2 Quake Engine und Source Ports

'Quake' ist ein von 'id Software' entwickeltes Videospiel, das 1996 vom Publisher 'GT Interactive' veröffentlicht wurde. Es ist vor allem bekannt dafür, dass es eines der ersten Videospiele war, das vollständig auf 3D-Meshes setzte, im Gegensatz zu bekannten anderen Ego-Shootern wie 'Doom', welches 2D-Sprites verwendete [Wik24].

Mit der fortschreitenden Grafiktechnologie und der Verbreitung von Grafikkarten entwickelten sich Ports und andere Versionen, die die Vorteile von hardwarebeschleunigtem Rendering nutzen konnten. 'id Software' selbst veröffentlichte einen offiziellen Port namens 'GLQuake', der auf die OpenGL Grafik API setzte. Die Veröffentlichung des Source Codes im Jahr 1999 führte außerdem dazu, dass weitere sogenannte 'Source-Ports' im Laufe der Jahre entwickelt und veröffentlicht wurden. Diese zielen darauf ab, die Kompatibilität mit Hardware und Betriebssystemen zu verbessern sowie Texturen und Effekte zu optimieren. Ein Beispiel für einen 'Source-Port' ist 'vkQuake', der die Vulkan Grafik API nutzt und somit für die ursprüngliche Modifikation kompatibel war [Gne16].



## 3 Schwerpunkte

### 3.1 Anforderungen and Kompatibilität

Zum Zeitpunkt dieser Arbeit (März 2024) befindet sich RTX Remix in der Beta-Phase mit der Version 2024.2.1. Es ist daher möglich, dass sich die Kompatibilität mit Spielen im Laufe der Zeit ändern kann. Gegenwärtig erfordert die Modifikation eines Spiels die Verwendung der Direct3D9-Grafik-API. Allerdings ist es für ältere Grafik-APIs wie Direct3D8 oder alternative APIs wie OpenGL möglich, die meisten Grafik-API-Aufrufe mithilfe von sogenannten Wrapper-DLLs in Direct3D9-Äquivalente umzuwandeln, mit denen Remix arbeiten kann.

Für eine optimale Kompatibilität muss das Spiel außerdem vollständig auf eine sogenannte 'fixed-function' Grafipeline setzen.

Hierbei handelt es sich um eine Grafipeline, bei der Texturen und Meshes mithilfe eines standardisierten Datenformats an die GPU gesendet werden. Diese Daten werden von der Remix-Runtime abgefangen, um die Spielszene zu rekonstruieren. Shader-Grafipelines werden hingegen nicht unterstützt, da die Datenstrukturen eines Shaders ein beliebiges Format annehmen können und daher nicht von Remix ohne Vorwissen verarbeitet werden können [nvi].

#### 3.1.1 Kompatibilität: Ursprüngliche Modifikation

Die ursprüngliche Modifikation wies ebenfalls einige Einschränkungen hinsichtlich der Spielekompatibilität auf. So war es erforderlich, dass die Engine des Spiels DirectX12 oder Vulkan als Grafik-API verwendete, da die Erweiterung für hardwarebeschleunigtes Ray Tracing zu diesem Zeitpunkt ausschließlich für die genannten APIs unterstützt wurden. Da die Implementierung auf Engine-Ebene erfolgte, war zudem der Zugriff auf den Quellcode erforderlich. Im Gegensatz zu Remix, gab es jedoch keine Einschränkung bezüglich der Grafipeline, da der Entwickler, mit Zugriff zum Sourcecode, optimalerweise bereits die Kontrolle über die gerenderten Elemente besitzt.

Das Spiel 'Quake' wurde aufgrund der Verwendung vom 'vkQuake' Source-Port, einer auf Vulkan basierten Engine, sowie der Möglichkeit, gemäß der GNU General Public License auf den Quellcode zuzugreifen ausgewählt. Allerdings erfüllten nur wenige Spiele diese Bedingungen, was zu einer stark eingeschränkten Auswahl führte.

#### 3.1.2 Kompatibilität: Remix Modifikation

RTX Remix bietet theoretisch die Möglichkeit, alle Direct3D9-Titel sowie potenziell ältere Spiele zu modifizieren, sofern ein entsprechender Wrapper verfügbar ist. In der Praxis ist

die Kompatibilität jedoch begrenzt, da Spiele trotz der Nutzung der passenden Grafik-API grafische Fehler aufweisen oder teilweise unspielbar sein können. Aus diesem Grund führen Nvidia und die RTX Remix Modding Community eine Liste mit kompatiblen Spielen und deren Kompatibilitätsstufen[mod].

Um die Kompatibilitätsproblematik zu veranschaulichen, wurden verschiedene Version von 'Quake' auf ihre Modifikationsmöglichkeiten hin untersucht 3.1.

| Engine            | API      | Kompatibilität | Anmerkung                              |
|-------------------|----------|----------------|----------------------------------------|
| MarkV Quake (DX9) | DirectX9 | Teilweise      | Spielabsturz bei neuem Spiel           |
| DirectQ 1.9.0     | DirectX9 | Nein           | Startet nicht                          |
| MarkV Quake (DX8) | DirectX8 | Teilweise      | Blackscreen / Sound & Eingabe möglich  |
| GLQuake           | OpenGL   | Teilweise      | Kein Path Tracing, da ungültige Kamera |
| Fitzquake         | OpenGL   | Nein           | Nicht funktionaler Wrapper             |
| Ironwail          | OpenGL   | Nein           | Nicht funktionaler Wrapper             |
| MarkV Quake (GL)  | OpenGL   | Nein           | Nicht funktionaler Wrapper             |
| Quakespasm        | OpenGL   | Nein           | Nicht funktionaler Wrapper             |

Tabelle 3.1: Kompatibilitätsliste

Die Untersuchung ergab, dass von insgesamt acht Versionen des Spiels 'Quake' lediglich drei erfolgreich mit der Remix Runtime gestartet werden konnten. Keines der OpenGL-Varianten mit Ausnahme von GLQuake, der offiziellen OpenGL-Portierung, konnten mithilfe des OpenGL-Wrapper [sou] gestartet werden. Die MarkV-Engine Variante zeigten zwar mehr Aktivität, waren jedoch ebenfalls nicht vollständig kompatibel. Die DirectX8-Variante beispielsweise lieferte kein Bild, Sound und Eingabe dagegen funktionierten, was auf eine potenzielle Kompatibilität hindeuten. Die DirectX9-Variante konnte zwar das Menü des Spiels korrekt anzeigen, führte jedoch zum Absturz des Spiels, sobald das erste Level gestartet wurde. Die OpenGL-Version hingegen lieferte zwar ein Bild und stürzte nicht ab, meldete jedoch eine ungültige Kamera, was dazu führte, dass das Path Tracing nicht ordnungsgemäß funktionierte.

Nach weiteren Überprüfungen wurde für diese Arbeit die MarkV-Engine (DX8), mit-samt des passenden 'DxWrapper' [Rie17], ausgewählt, da diese Version im Vergleich zu GLQuake technisch ausgereifter und performanter ist. Zudem wurde diese Version bereits mit einer Visual Studio Solution geliefert, was den Bau neuer Versionen erleichterte.

## 3.2 Texturen

In zahlreichen Anwendungen finden Texturen vielfältige Verwendungszwecke und dienen nicht ausschließlich der Texturierung von Meshes.

### 3.2.1 Texturen: Remix Modifikation

Aus diesem Grund ist es Ratsam als ersten Schritt die Kategorisierung von Texturen durchzuführen, die von der Remix Bridge in der Laufzeit abgefangen werden. Die Ka-

tegorisierung ist wichtig, um bestimmte Texturen vom Einfluss des Path Tracers auszuschließen. Beispielsweise sollten UI-Elemente entsprechend kategorisiert werden, damit die Lichtsimulation nicht beeinträchtigt wird. Gelegentlich werden Texturen als Teil eines Shaders verwendet, der sich in manchen Fällen auf eine unübliche Weise auf die Darstellung auswirken kann.

Diese Texturen können über ein separates Menü kategorisiert werden. Dabei ist zu beachten, dass die Texturen erst zur Laufzeit erscheinen, was bedeutet, dass jeder Bereich des Spiels zwangsläufig betreten werden muss, um den Draw Call zu verursachen, der die Textur in die Runtime lädt.

Die folgende Liste zeigt einige der Texturkategorien, die sich im ersten Level des Spiels finden lassen, sowie die Probleme, welche durch die Nicht-Kategorisierung auftreten können:

### User Interface Texturen

'Quake' verwendet eine Textur für sein Hauptmenü sowie zwei weitere Texturen für Zeichen und Schriftarten, die im Spiel verwendet werden. Die Kategorisierung dieser Texturen führt jedoch in diesem Fall zu einer ungültigen Kamera, was zum Scheitern des Path-Tracings führt. Die genaue Ursache für dieses Problem konnte leider nicht identifiziert werden. Es war jedoch möglich, die Texturen in diesem Fall unkategorisiert zu lassen, was während des Spiels zu keinen Problemen führte 3.1.

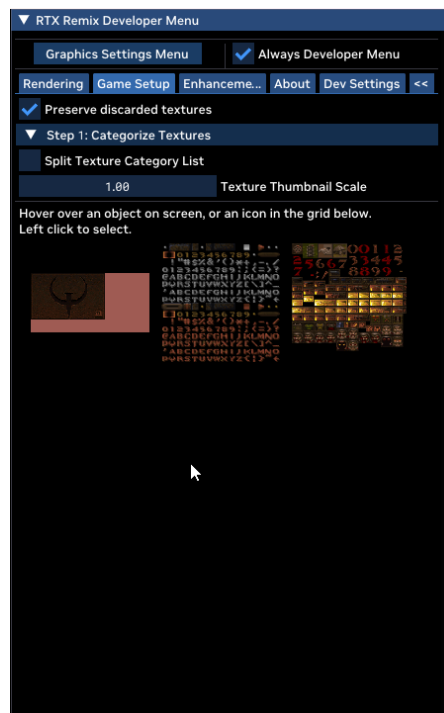


Abbildung 3.1: User Interface Texturen

## Mesh Texturen

Für 'Mesh Texturen', die zur Texturierung von Meshes verwendet werden, ist keine spezifische Kategorisierung erforderlich und können daher unkategorisiert bleiben 3.2.

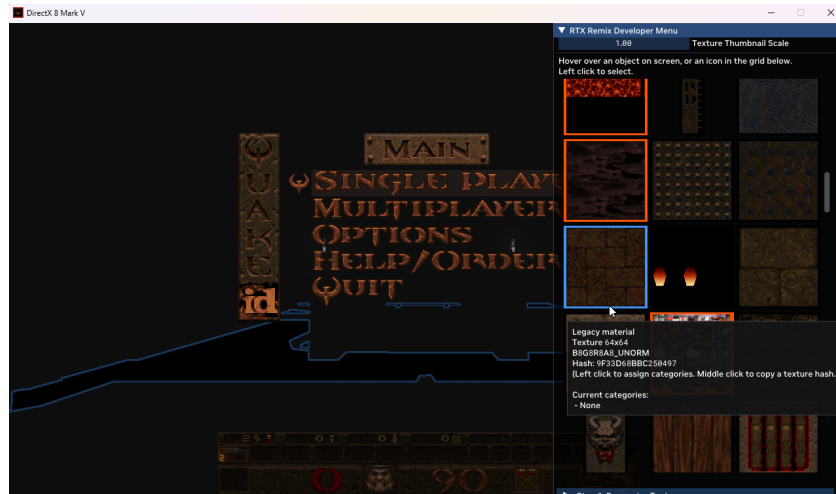


Abbildung 3.2: Mesh Texture und dazugehöriges Mesh (blauer Rahmen)

## Lightmap Texturen

In 'Quake' dienen Lightmap-Texturen zur statischen und dynamischen Beleuchtung von Meshes. Aufgrund der potenziellen Konflikte mit dem Path-Tracing-Lichtsystem ist es ratsam, diese Texturtypen entsprechend zu markieren. Ohne angemessene Kategorisierung führen Lightmaps zu dem unerwünschten Effekt, dass der Nutzer durch die Weltgeometrie schauen kann 3.3.

Gemäß dem Tooltip in der Remix Runtime ermöglicht die Kategorisierung, dass die Albedotextur eines Meshes ohne Einfluss der Lightmaptextur gelesen werden kann. In Quake führte diese Kategorisierung jedoch dazu, dass die betroffenen Meshes ohne Albedo-Textur angezeigt wurden. Dies resultierte in vollständig farblosen Meshes. 3.4

Als Alternative wurde die 'Ignore Texture' Kategorie ausgewählt, was die zuvor beschriebenen Probleme beseitigte. 3.5

## Lichttexturen

'Lichttexturen' ermöglichen es, Lichtquellen an Texturen anzubringen. Obwohl dieses Feature theoretisch verfügbar ist, scheint es in Quake nicht zu funktionieren, da keine Lichtquellen in der RTX-Anwendung erscheinen. Alternative Lösungsansätze werden in einem späteren Kapitel erörtert 3.6.

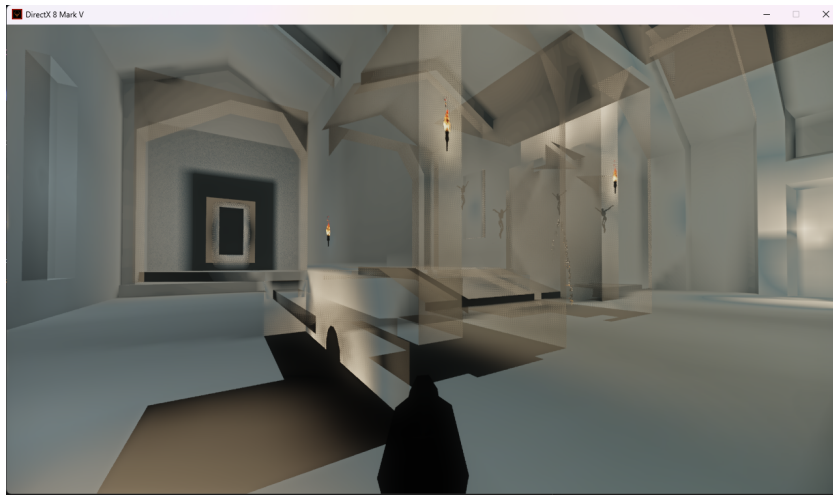


Abbildung 3.3: Nicht kategorisiert



Abbildung 3.4: Kategorie: 'Lightmap texture'

### 3.2.2 Texturen: Ursprüngliche Modifikation

Dieser Ansatz zur Texturkategorisierung unterscheidet sich grundlegend von der ursprünglichen Modifikation, bei der eine manuelle Bereitstellung einer Liste von Texturen erforderlich war, genauer gesagt von Sampler2D-Objekten, die im Ray Tracing Shader als Sampler2D-Buffer verwendet wurden. Zusätzlich lag die Verantwortung beim Benutzer, die Anordnung der Texturen im Buffer sowie deren Zuordnung zu den entsprechenden Meshes sicherzustellen. Eine inkorrekte Reihenfolge führte dazu, dass das Mesh die falsche Textur besaß. Zusätzlich musste man sicherstellen, dass die Berechnung der Baryzentrischen Koordinaten des Mesh-Polygons und das Datenformat der Texturen korrekt im Speicher abgelegt wurden, da es sonst zu einer fehlerhaften Anzeige kam. Diese Aufga-



Abbildung 3.5: Kategorie: 'Ignore texture'

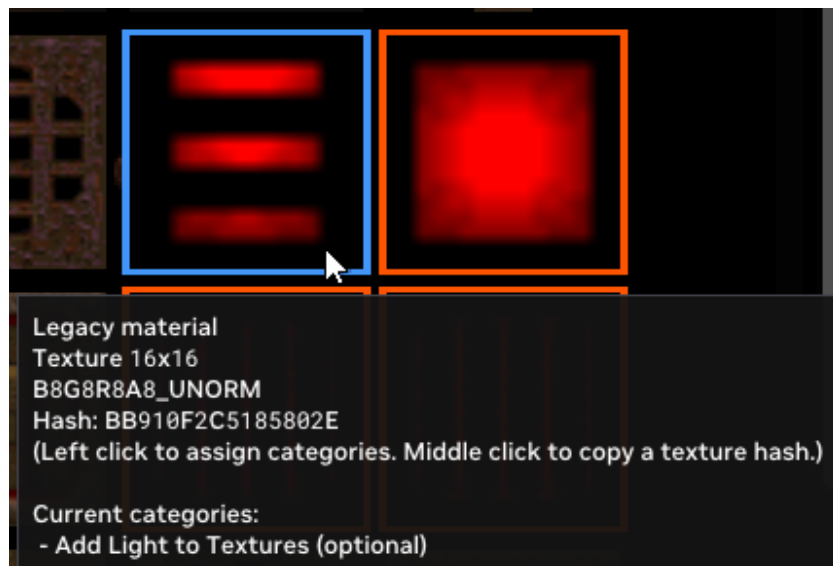


Abbildung 3.6: Emissive Lichttexturen

ben werden beispielsweise automatisch von RTX Remix übernommen.

Diese Vorgehensweise bietet sowohl Vor- als auch Nachteile. Ein wesentlicher Nachteil der ursprünglichen Implementierung bestand in der zeitaufwändigen Zuweisung der Meshes und Texturen, die ein detailliertes Verständnis der Grafikpipeline des Spiels erforderte.

Jedoch bot dieser Ansatz den Vorteil der umfassenden Kontrolle über die Texturen. So war es beispielsweise möglich, das Benutzeroberflächen- und Lightmap-Texturen vollständig aus dem Texturpool zu entfernen. Darüber hinaus konnten Lichttexturen, in Quake



auch als 'Fullbright' bezeichnet, als separater Texturbuffer an den Shader angehängt werden.

Anders ausgedrückt, erfolgte die Kategorisierung der Texturen durch das Anlegen verschiedenen Texturbuffer im Shader.

### 3.3 Kamera

In einigen Fällen kann es vorkommen, dass die RTX-Laufzeitumgebung zwar mit dem Spiel interagiert, jedoch keine sichtbaren Veränderungen bewirkt. Die Ursache hierfür liegt oft in einer ungültigen Kamera. Dies kann anhand der Kamerateypen in den 'Development Settings' der RTX-Laufzeitumgebung festgestellt werden. Bei einer gültigen Kamera sollten die Felder für die 'Main Camera' entsprechende Werte aufweisen, während bei einer ungültigen Kamera diese Felder leer bleiben 3.7. Eine valide Kamera ist unumgänglich, da sie der Ursprung der ausgesendeten Strahlen ist. Ohne eine valide Kamera ist deshalb das Path-Tracing nicht möglich.

Wie bereits im Kapitel 'Texturen' festgestellt wurde, handelte es sich hier eigentlich um einen Fehler bei der Kategorisierung einer UI-Textur, die aus unbekannten Gründen die Kamera als ungültig erachtete. Dies konnte allerdings nur durch Zufall herausgefunden werden, weshalb die Vermutung zuerst auf einen Fehler bei den Transformationsmatrizen oder der Wrapper-Klasse gesetzt wurde.

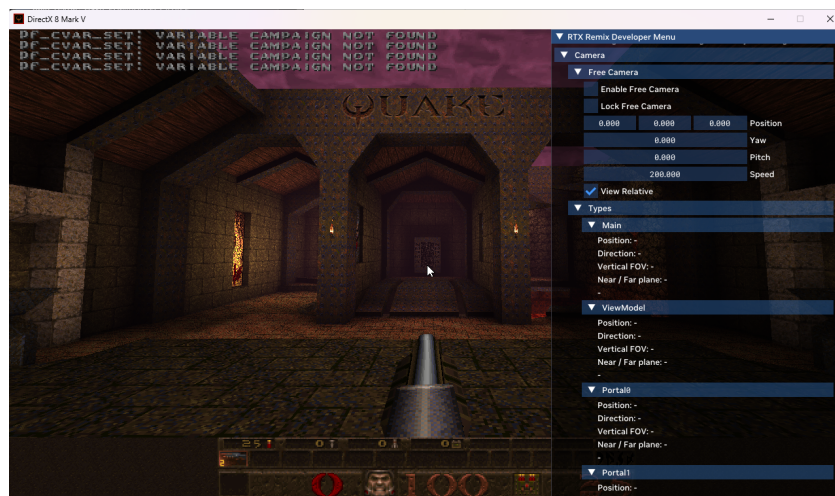


Abbildung 3.7: Ungültige Kamerawerte

Ursprünglich wurde vermutet, dass möglicherweise die Kameramatrizen als ungültig erachtet wurden. Diese Annahme konnte jedoch schnell widerlegt werden, da die Matrizen keine ungewöhnlichen Werte aufwiesen. Deshalb wurde die DX8 Wrapper-Klasse in Betracht gezogen, welche OpenGL-Befehle in DX8-Befehle umwandelt. Dabei wurden einige Inkonsistenzen mit der DX9 Wrapper-Klasse festgestellt, die möglicherweise den Fehler verursachten. Während OpenGL für die Kamera zwei Matrizen verwendet - eine Pro-

Projektionsmatrix und eine Model-View-Matrix - verwendet DirectX eine Projektionsmatrix sowie separate Model- und View-Matrizen. Hierfür setzen die Wrapper-Klassen eine der Matrizen, entweder 'D3DTS\_WORLD' oder 'D3DTS\_VIEW', als Identitätsmatrix und übertragen die Model-View-Matrix in die Andere. Allerdings gab es Uneinigkeiten zwischen den DX8- und DX9-Wrapper-Klassen über diese Einstellung, weshalb für Testzwecke die Identitätsmatrix jeweils für beide Transformationsmatrizen, 'D3DTS\_WORLD' und 'D3DTS\_VIEW', gesetzt wurde.

Im Entwicklermenü der Remix-Laufzeitumgebung kann im 'Game Setup' unter 'Parameter Tuning' festgelegt werden, in welcher der beiden Transformationsmatrizen sich die Model-View-Matrix befindet.

Jedoch gibt es im Falle von Quake einige Probleme:

1. Wenn die View-Matrix für die Model-View-Matrix verwendet wird und die World-Matrix als Identitätsmatrix bleibt, erkennt die Kamera Position und Ausrichtung korrekt, und die Welt wird korrekt dargestellt. Jedoch fehlen alle Alias-Modelle, einschließlich dynamischer Objekte wie das Viewmodel, Lichtquellen und Gegner. Diese werden nicht korrekt transformiert und befinden sich stattdessen alle an einem Ursprungspunkt. VIDEO: View Transform
2. Wenn hingegen die World-Matrix für die Model-View-Matrix verwendet wird und die View-Matrix als Identitätsmatrix bleibt, sind das Viewmodel sowie alle anderen dynamischen Objekte vorhanden. Allerdings ist die Kamera am Nullpunkt fixiert, und die Welt wird stattdessen transformiert. Dies an sich würde kein Problem darstellen, wenn das Anti-Culling ordnungsgemäß funktionieren würde. Bei genauerer Betrachtung der Welt ist zu erkennen, dass sie sich um das Viewmodel bewegt, was zu seltsamen Lichteffekten führt. VIDEO: World Transform

Um die Spielbarkeit zu gewährleisten wurde die zweite Variante gewählt, da das Fehlen der dynamischen Geometrien eine große Einschränkung auf das Spiel hat, auch wenn die Kamerawerte bei der ersten Variante richtig erscheinen.

## 3.4 Lichtquellen

Quake verwendet Lightmaps für die Beleuchtung von statischer als auch dynamischer Geometrie. Da das Ziel nun ist die ursprüngliche Beleuchtungsmethode zu ersetzen, müssen die Lightmaps entfernt werden. Doch die Entfernung der Lightmaps führt zu einem Problem, da neben ihnen keine anderen Lichtquellen im herkömmlichen Sinne über die Grafik-API gesetzt werden. Es gibt daher keine Referenzen oder Informationen über Lichtquellen sowie deren Eigenschaften wie Position, Richtung, Typ oder Farbe, womit die Spielszene ausgeleuchtet werden kann.

### 3.4.1 Lichtquellen: Ursprüngliche Modifikation

In der ursprünglichen Implementierung wurden statt herkömmlichen Lichtquellen, 'Fullbright'-Lichttexturen verwendet, um Lichtquellen zu simulieren. Sollte also eines der Strahlen

auf ein Mesh treffen, welches eine emissive Lichttextur enthielt, wurde zunächst mittels einer sogenannten 'Aktivierungsfunktion' überprüft, ob der Farbwert auf der Textur eine bestimmte Luminanzschwelle überschritt. War dies der Fall, wurde die getroffene Geometrie als Lichtquelle behandelt. Zudem sorgte die Aktivierungsfunktion dafür, dass nicht das gesamte Mesh als Lichtquelle fungiert, sondern nur die Bereiche, die tatsächlich fähig wären, Licht zu emittieren 3.8.

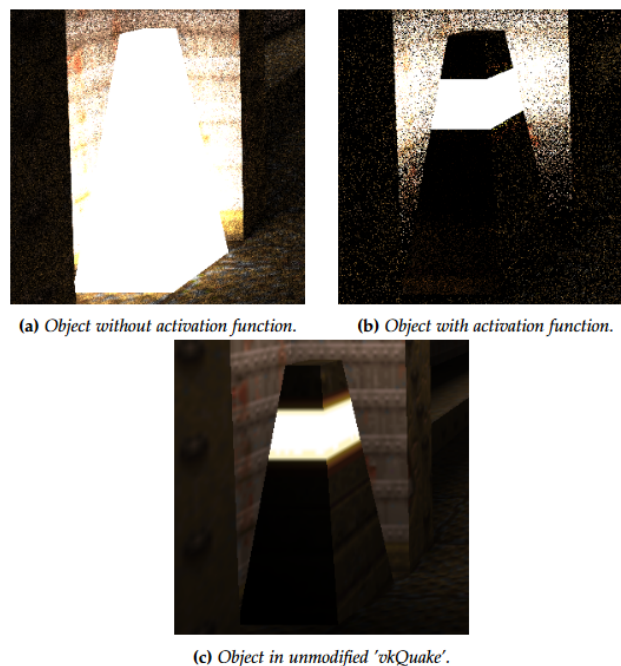


Abbildung 3.8: Vergleich Aktivierungsfunktion

Um sicherzustellen, dass eine solche Lichttextur Einfluss auf die Lichtsimulation hatte, musste sie jedoch von einem Strahl getroffen werden, was bei einem Monte-Carlo Path-Tracer nur zufällig geschehen kann. Der Einfluss der Lichtquelle wuchs also mit der Größe ihrer Oberfläche. Bei besonders kleinen Lichtquellen, wie beispielsweise Lampen, war der Einfluss daher besonders gering, vor allem da die Anzahl der Strahlen aus Performancegründen, normalerweise auf ein bis zwei Strahlen pro Pixel beschränkt ist. 3.9.

Eine Lösung für dieses Problem ist das 'Importance Sampling' oder auch 'Next Event Estimation', bei dem die Strahlen basierend auf einer Wahrscheinlichkeitsberechnung häufiger in Richtung einer einflussreichen Lichtquelle gesendet werden, um den Einfluss von Lichtquellen bei geringen Samplingraten signifikant zu erhöhen. Diese Methode erfordert jedoch Kenntnisse über Eigenschaften wie beispielsweise die Position im Voraus, was in Quake nicht ohne weiteres umsetzbar war, weshalb das 'Importance Sampling' in der ursprünglichen Modifikation gänzlich fehlte.

In einem sehr frühen Prototypen wurde versucht, in einer benutzerdefinierten Datenstruktur die Positionsdaten einer Lichttextur zu erfassen, die auf dem Mesh liegt. Da die

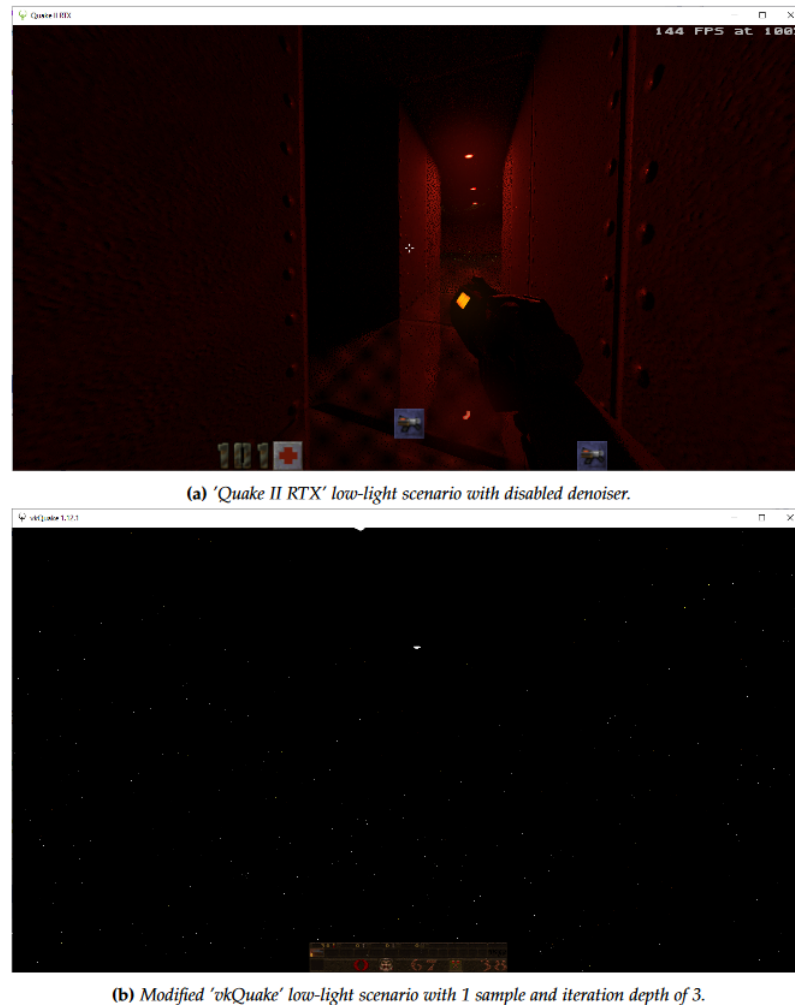


Abbildung 3.9: Lichteinfluss mit (oben) und ohne (unten) Lighttracing bei kleiner Lichtquelle.

Lichttextur, die auf dem Mesh liegt, meist nur an wenigen Stellen der Textur hell genug ist, war es außerdem erforderlich, den möglichen Positionsraum einzuschränken. Die ursprüngliche Idee war, zu jeder Textur eine Bounding Box zu erstellen, die den beleuchteten Bereich einschränkt. Eine Alternative zur Bounding Box wäre, die Textur in eine diskrete Anzahl von Kacheln aufzuteilen und für jede Kachel einen Durchschnittsfarbtton sowie ihren Luminanzwert festzuhalten.

Um die Idee zu verdeutlichen, nehmen wir ein Fackelmodell und seine Textur als Beispiel. Da nur der Flammenbereich der Textur in der Lage wäre, Licht zu emittieren, würde man hier nur den Bereich der Textur einschränken, der die Flamme enthält. Anschließend würde man daraus eine 3D-Bounding Box um das Mesh, das die Flammentextur enthält,

oder eine Liste diskreter Weltpositionen erstellen, die die Flamme im Mesh einschränken, sodass das Importance Sampling Lichtquellen-Positionen zur Auswertung hat.

### 3.4.2 Lichtquellen: Remix Modifikation

Ein ähnliches Problem tritt auch bei RTX Remix auf. Da keine Lichtquellen über die Grafik-APIs gesetzt werden, erkennt die RTX Remix Bridge auch keine entsprechenden Aufrufe und stellt folglich keine Lichtquellen in der Remix-Anwendung dar. Es gibt jedoch verschiedene Möglichkeiten, dieses Problem zu umgehen.

#### Stage Light

Eine Möglichkeit besteht darin, eine sogenannte 'Stage Light' an ein Mesh anzuknüpfen, um die fehlenden Lichtquellen zu kompensieren. In diesem Beispiel wurde das Fackelmodell, welches sich im Hub-Level befindet, verwendet.

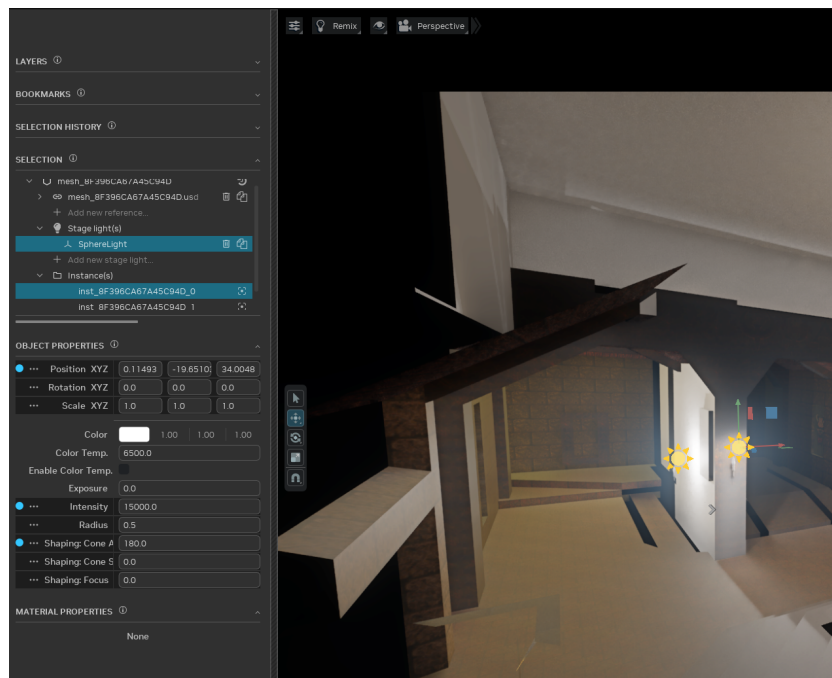


Abbildung 3.10: Lichtquellen und deren Eigenschaften

Bei dieser Herangehensweise treten jedoch einige Probleme auf. Standardmäßig sind alle Meshes undurchsichtig. Im Szenario des Fackel-Modells im ersten Level ist das Stage-Light bei der Initialisierung im Zentrum des Meshes positioniert. Um den Raum angemessen zu beleuchten, müsste die Lichtquelle außerhalb des Meshes platziert werden. Dies hätte jedoch zur Folge, dass das Mesh selbst - in diesem Fall die Fackel - einen eigenen Schatten wirft, was in bestimmten Situationen unerwünscht ist.

Um dieses Problem zu umgehen, besteht die Möglichkeit, das Mesh als transluzent zu kennzeichnen. Dadurch könnte das Licht von innen das Mesh durchdringen und den Raum beleuchten. Dies mag beispielsweise bei Lampen Sinn ergeben, jedoch wirkt es bei einer Fackel unpassend. Zudem sind in diesem Fall sowohl die Flamme als auch der Griff Teil desselben Meshes. Das hätte zur Folge, dass die gesamte Fackel durchsichtig wäre.

Ein weiteres spezifisches Problem bei der Fackel ist, dass sie mithilfe von Vertex-Animation animiert wird. Dies führt jedoch dazu, dass jede Animationsphase ein eigenes Mesh generiert. Da die Lichtquelle einem bestimmten Mesh zugewiesen ist, existiert die Lichtquelle nur für die bestimmte Animationsphase, was zu einem starken Flackern führt VIDEO: Vertexanimation Lichtflackern. In solchen Fällen ist es nicht möglich, Lichtquellen zu verwenden.

### Emissive Textures

Als alternative Methode zur Lichtquelle besteht die Möglichkeit, die zugrunde liegende Textur als emissiv zu markieren. Diese Vorgehensweise beseitigt das Problem des flackern des Lichts, da die Emissionseigenschaft auf das darunterliegende Material und nicht auf das Mesh angewendet wird. Der Nachteil der emissiven Texturen besteht jedoch darin, dass die Einstellungen im Vergleich zur Verwendung einer Lichtquelle begrenzter sind. Akutell wird die Emissionsfarbe gleichmäßig auf das gesamte Mesh übertragen. Um dies auszugleichen, kann jedoch eine Maske verwendet werden, um nur einen Teil des Meshes als emissiv zu markieren. Diese Maske ähnelt der Aktivierungsfunktion in der ursprünglichen Version, mit dem Unterschied, dass die Maske als vorbereitete Textur verwendet werden muss, anstatt dass sie zur Laufzeit berechnet wird 3.11.

In RTX Remix bietet sich ein weiterer Vorteil durch die Integration von RTX Dynamic Illumination (RTXDI), einem SDK, das verschiedene Techniken zum schnellen Sampling von Lichtquellen ermöglicht, einschließlich emissiver Texturen. Es nutzt einen ähnlichen Ansatz wie im Abschnitt zuvor beschrieben 3.4.1, jedoch in einem technisch fortgeschrittenen und umfassenderen Zustand. RTXDI verwendet ebenfalls einen Preprocessing-Schritt, um emissive Lichtpartien in der Textur diskret aufzuteilen und zu identifizieren. Statt einer Bounding Box oder eines Kachelsystems werden dabei die Mesh-Triangles genutzt, die in einem Lichtcache gespeichert werden. Darüber hinaus kommen Techniken wie 'Reservoir-based Spatio-Temporal Importance Resampling' (ReSTIR) und 'Reservoir-based Grid Importance Resampling (ReGIR)' zum Einsatz, um den Einfluss des Lichtsamplings zusätzlich zu erhöhen, selbst bei einer sehr hohen Anzahl verfügbarer Lichtquellen [Wym20] [Pan21].

Das Resultat ist, dass emissive Lichtquellen, im Vergleich zur ursprünglichen Version, wesentlich mehr Einfluss auf die Beleuchtung haben.

## 3.5 Culling & Beschleunigungsstrukturen

Das Culling-Verfahren ist zwar gut, um die Leistung eines Spiels erheblich zu steigern, führt aber leider zu fehlerhaften Lichtberechnungen, da Geometrie ausgeblendet wird,





Abbildung 3.11: Emissive Texturen als Lichtquelle

die eigentlich sichtbar sein müsste.

### 3.5.1 Culling: Ursprüngliche Modifikation

Die Methode 'R\_DrawWorld' ist dafür verantwortlich, für jeden Frame alle Polygone zu berechnen, die für die Darstellung auf dem Bildschirm relevant sind, und diese zu markieren. Dabei werden sowohl statische als auch dynamische Geometrien berücksichtigt. Neben dem Frustum-Culling nutzt Quake ein System namens PVS (partially visible sets), um zu überprüfen, ob die Geometrie von der Position des Spielers aus sichtbar ist, indem es die Meshes zusätzlich in Polygonteile zerlegt.

In der ursprünglichen Implementierung wurde das PVS für statische Geometrie umgangen, indem die gesamte Level-Geometrie vor Spielbeginn in einen Buffer geladen wurde. Für dynamische Objekte war dieser Ansatz jedoch nicht möglich oder nicht empfehlenswert, da das Schreiben in den Buffer pro Frame sehr rechenintensiv war. Stattdessen wurde weiterhin die Draw-List für dynamische Elemente verwendet, die pro Frame verfügbar war und alle Daten für die Darstellung der Geometrie enthielt. Das Deaktivieren des Frustum-Cullings war für dynamische Geometrie zwar möglich, jedoch erwies sich das PVS-System als zu komplex, um es vollständig zu umgehen, was dazu führte dass dynamische Objekte nachwievor herausgeschnitten wurden.

### 3.5.2 Culling: Remix Modifikation

RTX Remix bietet für dieses Problem ein Anti-Culling-System für Objekte und Lichtinstanzen, um Lichtartefakte durch fehlende Geometrie zu minimieren. Das grundlegende Prinzip besteht darin, zu überprüfen, ob im vorherigen Frame unsichtbare Geometrie, im aktuellen Frame sichtbar ist. Diese Geometrie wird dann zusammen mit der aktuellen sichtbaren Geometrie verwendet, um einen Anti-Culling-BVH (Bounding Volume

Hierarchy) zu erstellen.

Das Anti-Culling-System hat jedoch einige Einschränkungen. Es kann beispielsweise keine Draw-Calls vorhersagen, wodurch das System erst im Laufe der Zeit stabilisiert wird. Culling-Artefakte sind also bei neu auftretenden Meshes mindestens einmal sichtbar. Zudem kann es nicht vorhersagen, welches Culling-System das Spiel verwendet, was zu einigen Unzulänglichkeiten führen kann.

In der Praxis behält das Anti-Culling-System die meiste Geometrie gut, aber es gibt gelegentlich 'Light Leaks'. Besonders in Spielen wie Quake, sind diese Artefakte bemerkbar, da generell wenige Lichtquellen vorhanden sind. Ein möglicher Grund für das Versagen des Anti-Culling-Systems könnte sein, dass das PVS-System ganze Geometrien zusätzlich in weitere Untergeometrien teilt, die sich jedoch überschneiden. 3.12 3.13 Dies könnte dazu führen, dass das Anti-Culling-System möglicherweise nur eines der überschneidenden Meshes behält, was zum Fehlverhalten führt. Diese Vermutung ist jedoch spekulativ und bedarf weiterer Untersuchungen. Tatsache ist jedoch, dass das Anti-Culling-System in Quake nicht einwandfrei funktioniert.

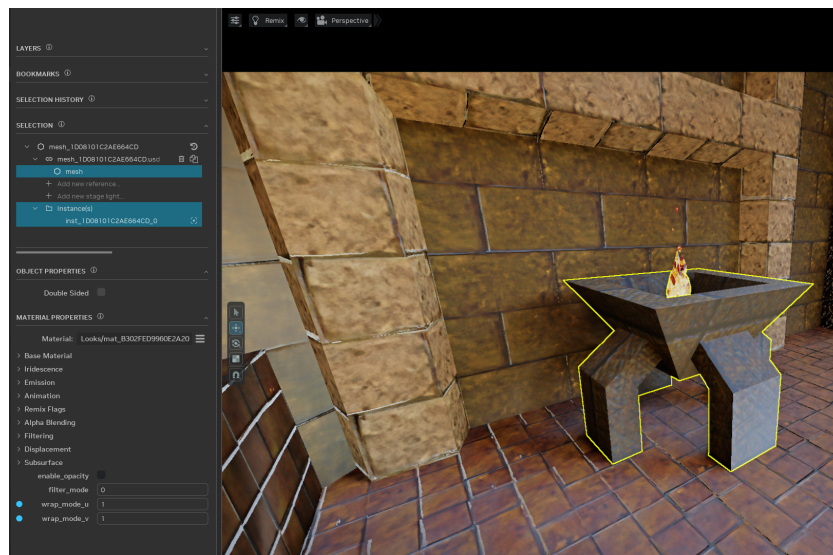


Abbildung 3.12: Unveränderte Geometrie

### 3.5.3 Beschleunigungsstrukturen: Ursprüngliche Modifikation

Die Problematik des Cullings betrifft zugleich auch die Erstellung der Beschleunigungsstrukturen. In der ursprünglichen Modifikation wurde eine zweistufige BLAS-Struktur verwendet. Wie bereits beschrieben war es möglich die gesamte statische Levelstruktur vor Beginn eines Level in einem Uniform-Buffer zu speichern, womit die gesamte erste BLAS erstellt wurde 3.14. Für dynamische Objekte wurde eine separate dynamische BLAS erstellt 3.15, die in jedem Frame aktualisiert wurde. Ein Vorteil dieses Ansatzes liegt darin, dass ein Großteil der verwendeten Geometrie im Voraus bekannt ist, was die



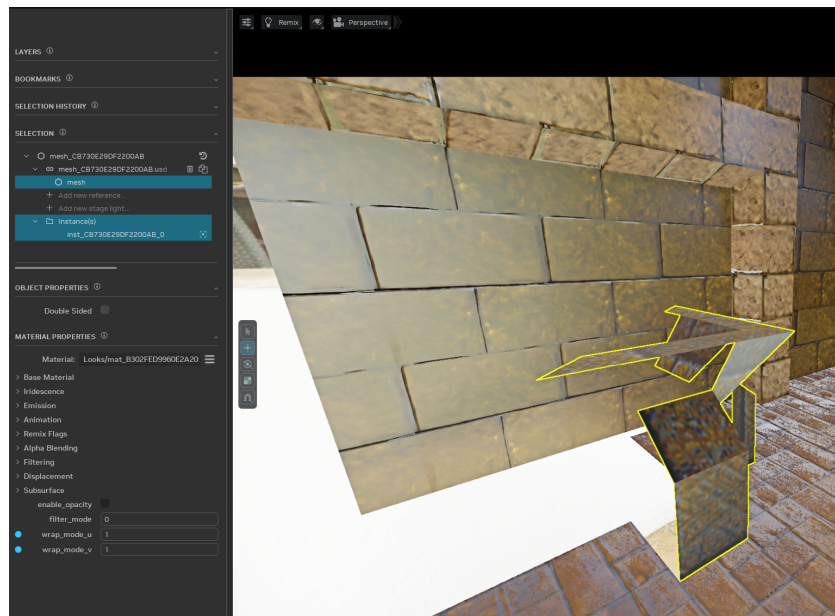


Abbildung 3.13: Culled PVS Geometrie

Erstellung der Beschleunigungsstrukturen vergleichsweise kostengünstig macht. Auch die Erstellung der dynamischen BLAS war vergleichsweise günstig, da die Anzahl der dynamischen Geometrien recht gering war und zudem die einzelnen Geometrien sich im selben Buffer befanden.

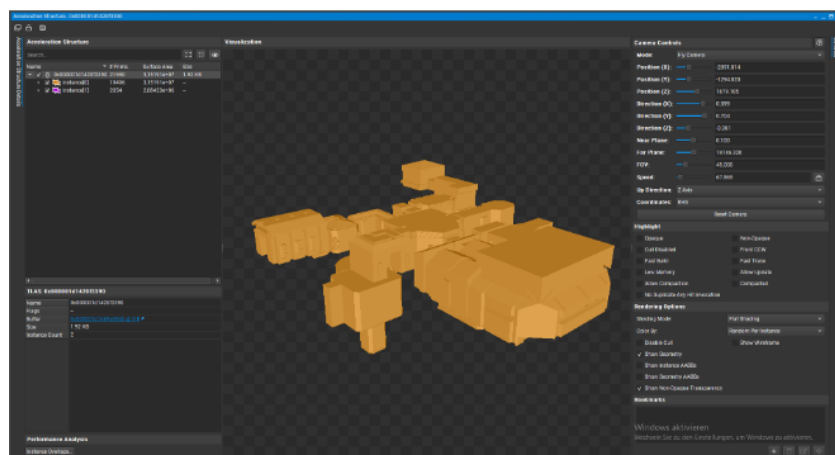


Abbildung 3.14: Statische BLAS

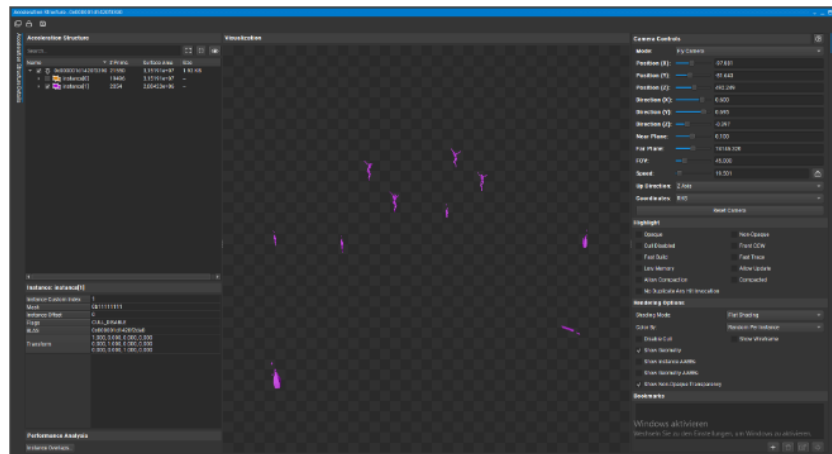


Abbildung 3.15: Dynamische BLAS

### 3.5.4 Beschleunigungsstrukturen: Remix Modifikation

In RTX Remix gestaltet sich dieser Prozess jedoch komplizierter, da im Voraus nicht einfach zu berechnen ist, wie die Geometrie im Speicher aufgeteilt wird. Auf direkter Anfrage an einen der Entwickler von RTX Remix, Mark Henderson, wird seiner Aussage nach zu jedem einzelnen Draw Call eine BVH erstellt, die dann zu einem TLAS (Top-Level Acceleration Structure) zusammengeführt wird.

Dies hat Implikationen für die allgemeine Performance des Spiels, da die Erstellung einer BLAS- beziehungsweise TLAS-Instanz eine nicht unerhebliche Zeit benötigt. In der ursprünglichen Implementierung wurde eine durchschnittliche Rechenzeit von 1,12 ms für das statische BLAS und 0,4 ms für die dynamische BLAS erreicht [Nov22]. Zu beachten ist jedoch, dass es sich dabei um Rechenzeiten eines Debug-Builds handelte, die über Nvidia Nsight ausgelesen wurden. Dies bedeutet, dass die Rechenzeit für einen Release-Build sicherlich optimiert wäre. Dennoch lässt sich festhalten, dass allein die Instanziierung einer Beschleunigungsstruktur einen gewissen Overhead mit sich bringt, der vorallem in Videospielen nicht gering ist.

Der Code, der für die Erstellung der Beschleunigungsstrukturen verantwortlich ist, verweist allerdings auf einen Pooling-Mechanismus, der dafür sorgt, dass häufig auftretende Geometrie wiederverwendet wird, um die Erstellung neuer BVH-Instanzen zu reduzieren [Gam23]. In der Praxis war die Performance ebenfalls vergleichbar mit der ursprünglichen Version, jedoch basiert dies lediglich auf einem subjektiven Eindruck. Konkrete Daten liegen hierzu nicht vor.

## 3.6 Materialien

### 3.6.1 Materialien: Ursprüngliche Modifikation

Quake verwendet ausschließlich diffuse Texturen zur Texturierung von Meshes und verfügt nicht über ein Materialsystem.

Daher wurde in der ursprünglichen Implementierung auf den Einbau eines Materialsystems verzichtet. Dies hätte die Entwicklung entsprechender Material Maps für viele Texturen sowie umfangreiche Änderungen am Code erfordert. Zeit und Ressourcen dafür waren jedoch nicht verfügbar.

Im Path Tracing wird zur Simulation von Materialien, insbesondere opaken Oberflächen, eine sogenannte 'Bidirectional Reflectance Distribution Function' (BRDF) verwendet. Diese Funktionen bestimmen für einen gegebenen Einfallswinkel auf ein Material einen passenden Ausfallswinkel. Sehr diffuse Materialien reflektieren beispielsweise Licht fast gleichmäßig von ihrer Oberfläche.

Um den ursprünglichen diffusen Look beizubehalten, wird im Hit-Shader des Path Tracers eine gleichmäßig zufällige Richtung in der Hemisphäre definiert, die von der Normalen des Treffers ausgeht 3.16. Diese Methode beschränkt jedoch die Unterstützung von spiegelnden, metallischen oder transluzenten Materialien.

```

1  const float theta = M_PI * 2 * rand(seed); // Random in [0, 2pi]
2  const float u     = 2.0 * rand(seed) - 1.0; // Random in [-1, 1]
3  const float r     = sqrt(1.0 - u * u);
4
5  hitPayload.direction = geometricNormal + vec3(r * cos(theta), r * sin(theta), u);
6  hitPayload.origin = position + 0.0001 * hitPayload.direction;
7  hitPayload.contribution *= luminance + albedo;

```

Abbildung 3.16: Berechnung des random uniform hemisphere samplings

### 3.6.2 Materialien: Remix Modifikation

Das Materialhandling bei RTX Remix ist umfangreicher und bietet die Möglichkeit, individuelle Meshes und Texturen durch weitere Texturen zu erweitern oder komplett zu ersetzen. Zudem ermöglicht RTX Remix die Generierung von PBR-Materialien mithilfe von KI.

Ein einfaches Beispiel zeigt die Ersetzung der Bodentextur mithilfe von RTX Remix. Dabei werden zunächst passende Materialien mithilfe des PBR-Materialgenerators erstellt, der alternative Diffuse-Texturen sowie Normal- und Roughness-Maps für die ausgewählte Textur generiert. 3.17

Die generierten Texturen müssen im 'Ingestion'-Tab in ein USD-Format umgewandelt werden, um die Original-Meshes oder Texturen zu ersetzen. Neben Albedo-, Normal- und Roughness-Texturen bietet RTX Remix die Möglichkeit, Emissive-Mask-, Metallic- und Height-Maps hinzuzufügen, falls eigene Texturen entwickelt werden. Nach Abschluss der

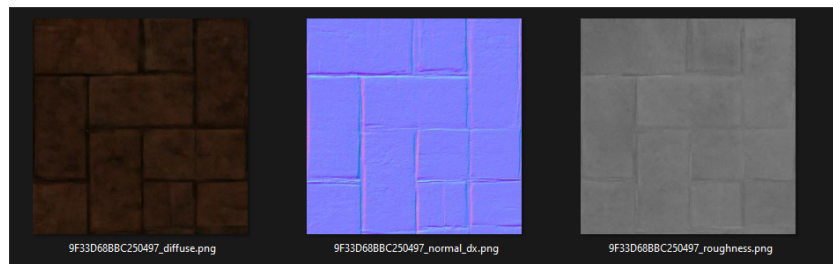


Abbildung 3.17: KI-Generierte Diffuse, Normal und Roughness Maps

Konvertierung können die Texturen genutzt werden, um die ursprünglichen Texturen zu ersetzen.3.18 3.19



Abbildung 3.18: Standard Material

Dadurch ist es selbst Nicht-Künstlern möglich, verbesserte Materialien zu entwickeln, die die Vorteile des Path Tracings besser nutzen können. Allerdings besteht kein direkter Einfluss auf die von der KI entwickelten Texturen. Es kann daher vorkommen, dass die generierten Texturen nicht dem Stil der Originaltextur entsprechen. Auch besteht derzeit keine Möglichkeit, den Einfluss der einzelnen Texturen manuell anzupassen. Es ist zwar möglich, einen Scale und Bias für PBR-Materialien festzulegen, dieser gilt jedoch für alle Materialien und sollte laut ihrer Toolbox nur für Debugging-Zwecke verwendet werden.

Eine Alternative besteht darin, ohne jegliche Ingestion die Eigenschaften des Standardmaterials zu manipulieren, das für jedes Mesh gilt, ähnlich dem, was in der Modifikation implementiert wurde. VIDEO: Material Override.



Abbildung 3.19: Material überschrieben mit KI-Generierten PBR Texturen



## 4 Zusammenfassung

RTX Remix bietet eine kompetente Lösung für viele der beschriebenen Kernbereiche. Aufgrund der Anforderung, dass Remix mit einer Vielzahl verschiedener Spiele funktionieren muss, ist jedoch nicht jede Lösung optimal. Zum Beispiel sind die Erzeugung von Beschleunigungsstrukturen und das Culling nicht immer perfekt, weshalb eine nachträgliche Modifizierung des Originalspiels erforderlich ist. Trotzdem profitiert Remix von fortschrittlichen Techniken wie RtxDi und DLSS, die eine hochentwickelte Path-Tracing-Funktionalität ermöglichen, die schwer selbst zu implementieren wäre.

Da sich Remix noch in der Beta-Phase befindet, gibt es viele Aspekte, die noch nicht vollständig oder überhaupt nicht funktionieren. Auch ist nicht immer klar, welche Ergebnisse bestimmte Aktionen haben, wie beispielsweise bei der Kamerasteuerung. Oftmals kann der Nutzer nur Mutmaßen, was hinter den Kulissen der Anwendung wirklich geschieht. Die Wahl von 'Quake' als Testspiel erwies sich dabei als besonders anspruchsvoll. Möglicherweise ist Quake aufgrund seiner älteren Struktur, die ursprünglich auf einem OpenGL-Port basiert, nicht optimal für eine Modifizierung geeignet.

Insgesamt stellt RTX Remix ein leistungsfähiges Werkzeug dar, das jedoch keine sofortige Lösung bietet. Der Schwerpunkt liegt, meiner Meinung nach darauf, primär die zugrunde liegenden Assets und Materialien zu verbessern. Die Lösung von Kompatibilitätsproblemen kann sich jedoch als äußerst schwierig oder sogar unmöglich erweisen und hängt stark von dem Zugriffslevel ab, den man auf die zu modifizierende Software oder das Spiel hat.





# Literaturverzeichnis

- [Gam23] GAMEWORKS, NVIDIA: *dxvk-remix/src/dxvk/rtx\_render/rtx\_accel\_manager.cpp at main · NVIDIAGameWorks/dxvk-remix* — *github.com*. [https://github.com/NVIDIAGameWorks/dxvk-remix/blob/main/src/dxvk/rtx\\_render/rtx\\_accel\\_manager.cpp](https://github.com/NVIDIAGameWorks/dxvk-remix/blob/main/src/dxvk/rtx_render/rtx_accel_manager.cpp), 2023. [Accessed 01-04-2024].
- [Gne16] GNEITING, AXEL: *vkQuake*. <https://github.com/Novum/vkQuake>, 2016.
- [mod] *Video Games* — *moddb.com*. <https://www.moddb.com/rtx>. [Accessed 28-03-2024].
- [Nov22] NOVIKOV, RUSLAN: *Implementing a hardware-accelerated path tracing renderer in 'Quake'*, 2022.
- [nvi] NVIDIA RTX Remix *rtx\_remix 2024.2.1 documentation* — *docs.omniverse.nvidia.com*. [https://docs.omniverse.nvidia.com/kit/docs/rtx\\_remix/latest/index.html](https://docs.omniverse.nvidia.com/kit/docs/rtx_remix/latest/index.html). [Accessed 28-03-2024].
- [NVI23] NVIDIAGAMEWORKS: *RTX Remix*. <https://github.com/NVIDIAGameWorks/rtx-remix>, 2023.
- [Pan21] PANTELEEV, ALEXEY: *Implementing Ray-Traced Lighting with RTX-DI NVIDIA On-Demand* — *nvidia.com*. <https://www.nvidia.com/en-us/on-demand/session/gtcspring21-s32718/?playlistId=playlist-302fa2d9-2b01-491b-b6c5-fc6925866f35>, 2021. [Accessed 29-03-2024].
- [Rie17] RIEDLINGER, ELISHA: *DxWrapper*. <https://github.com/elishacloud/dxwrapper>, 2017.
- [sou] *GLDirect* — *sourceforge.net*. <https://sourceforge.net/projects/gldirect/>. [Accessed 28-03-2024].
- [Wik24] WIKIPEDIA: *Quake (video game)* — *Wikipedia, The Free Encyclopedia*. [http://en.wikipedia.org/w/index.php?title=Quake%20\(video%20game\)&oldid=1215773432](http://en.wikipedia.org/w/index.php?title=Quake%20(video%20game)&oldid=1215773432), 2024. [Online; accessed 28-March-2024].
- [Wym20] WYMAN, CHRIS: *Lighting Scenes with Millions of Lights Using RTX Direct Illumination | NVIDIA Technical Blog* — *developer.nvidia.com*. <https://developer.nvidia.com/blog/lighting-scenes-with-millions-of-lights-using-rtx-direct-illumination/>, 2020. [Accessed 29-03-2024].



# Annex

Github Repository

Google Drive: Projektdateien